Software Requirements Specification

for

UniPool

Version 1.0

Prepared by

Group #16 **Group Name: Undefined_Behaviour**

Aalok	240011	aalok24@iitk.ac.in
Alankrit Gupta	220110	alankritg22@iitk.ac.in
Anubhav Rana	210162	anubhavr21@iitk.ac.in
Aryan Nigam	220227	naryan22@iitk.ac.in
Bhavishya Gupta	220295	bhavishyap22@iitk.ac.in
Gagandeep Kolambe	220393	gagandeep22@iitk.ac.in
Harshit Gupta	220439	gharshit22@iitk.ac.in
Karan Kumar	230533	karanm23@iitk.ac.in
Naman Mohan Singh	230678	mohans23@iitk.ac.in
Pavi Agarwal	220762	apavi22@iitk.ac.in

Course: CS253

Mentor TA: V Arvind

Date: 21 January 2025

CONTENTS II

REVISIONS III

1 INTRODUCTION 1

1.1 PRODUCT SCOPE 1
1.2 INTENDED AUDIENCE AND DOCUMENT OVERVIEW 1
1.3 DEFINITIONS, ACRONYMS AND ABBREVIATIONS 1
1.4 DOCUMENT CONVENTIONS 2
1.5 REFERENCES AND ACKNOWLEDGMENTS 2

2 OVERALL DESCRIPTION 3

2.1 PRODUCT OVERVIEW 3
2.2 PRODUCT FUNCTIONALITY 4
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS 4
2.4 ASSUMPTIONS AND DEPENDENCIES 6

3 SPECIFIC REQUIREMENTS 7

3.1 EXTERNAL INTERFACE REQUIREMENTS 7
3.2 FUNCTIONAL REQUIREMENTS 15
3.3 USE CASE MODEL 21

4 OTHER NON-FUNCTIONAL REQUIREMENTS 33

4.1 PERFORMANCE REQUIREMENTS 33
4.2 SAFETY AND SECURITY REQUIREMENTS 34
4.3 SOFTWARE QUALITY ATTRIBUTES 35

APPENDIX A – DATA DICTIONARY 38

APPENDIX B – GROUP LOG 42

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Anubhav Rana Pavi Agarwal Naman Mohan Singh Karan Kumar Gagandeep Kolambe Aalok Alankrit Gupta Aryan Nigam Bhavishya Gupta Harshit Gupta	Preliminary draft	20/01/25

1 Introduction

1.1 Product Scope

UniPool is a carpooling app designed for the IIT Kanpur community to simplify travel to shared destinations like airports and stations. The platform allows students to save money and reduce carbon emissions by either hosting rides as "Leaders" or joining them as "Poolers." With built-in features like real-time chat, ride-history trust scores, and secure verification, UniPool replaces disorganized messaging groups with a seamless, reliable, and eco-friendly transportation network

1.2 Intended Audience and Document Overview

This Software Requirements Specification (SRS) is mainly intended for the people connected to the project, namely, the course instructor, our TA, our team, and the team that intends to beta-test the application.It provides a comprehensive framework for the implementation and validation of the UniPool application, catering to the specific needs of each stakeholder.

As an evolving document, this SRS is subject to continuous change and refinement. It serves as a foundational guide that will be updated as development progresses and the product vision gains clarity.

By maintaining this central reference, all parties- from technical teams to end-users-stay aligned on the platform's goals, security standards, and functional requirements.

1.3 Definitions, Acronyms and Abbreviations

Admin	A registered user of the app with special privileges
API	Application programming interface
Firebase	Google's Backend-as-a-Service platform providing authentication, database, and storage services
Firestore	Cloud Firestore – NoSQL document database provided by Firebase
Flutter	Cross-platform mobile application development framework by Google
IEEE	Institute of Electrical and Electronics Engineers
IIT Kanpur/ IITK Institute	Indian Institute of Technology, Kanpur
Leader	A user who creates and posts a ride, planning to book a cab for a specific route and willing to share with others
No SQL	Non-relational database management system

Pooler	A user who browses and joins existing rides posted by Leaders
Registered User	A user of the app who has signed up using email and password
IP address	Internet Protocol address
OTP	One-time password
Registered user	A user of the app who has signed up using his Institute credentials
SDK	Software development kit
UI	User interface
UUID	Universally Unique Identifier
UX	User experience
XML	Extensible Markup Language

1.4 Document Conventions

These software requirements specifications were prepared following the software and systems engineering standards of the Institute of Electrical and Electronics Engineers (IEEE) cited below.

1. Text in this document is laid out in Arial with body text having font size 11 pt.
2. The document maintains a 1” margin and is single-spaced throughout.
3. Section titles use font size 18 pt. and are shaded. Top-level headings use font size 14 pt., while second-level headings use 12 pt.
4. Unordered list items are prefixed with circular bullets.
5. Comments are italicised and keywords are in bold.

1.5 References and Acknowledgments

1.5.1 References:

Style guide (for this document): IEEE Software Requirements Specifications Guide

1.5.2 Acknowledgement:

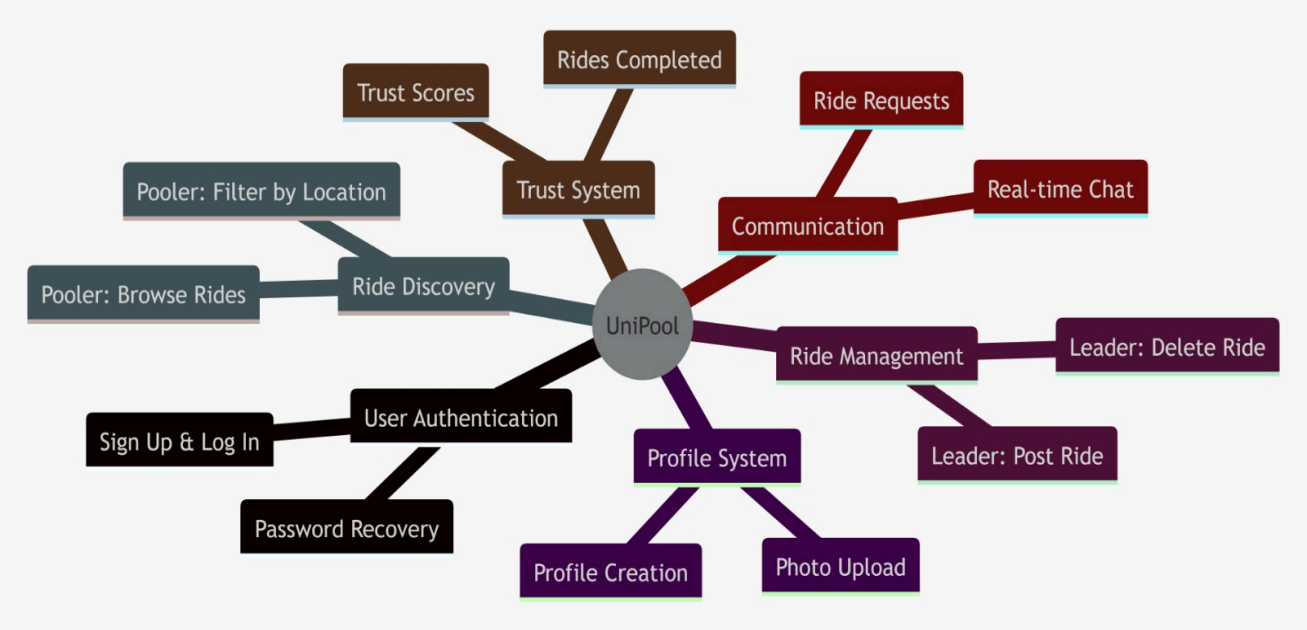
We are highly indebted to Professor Indranil Saha for his excellent lectures and notes, which largely provided the basis for our system design, and for diligently resolving all our queries regarding various functional and organizational aspects of the software. We are also obliged to our Teaching Assistant,V Arvind, for his timely assistance whenever required.

2 Overall Description

2.1 Product Overview

UniPool is a self-contained mobile application designed exclusively for the student community at IIT Kanpur to facilitate collaborative ride-sharing. The application serves as a digital platform that connects students who share common travel destinations, enabling them to coordinate shared transportation, reduce individual travel costs, and contribute to environmental sustainability.

The application operates as a standalone system that bridges the gap between students who have planned travel and those seeking rides to similar destinations. Unlike traditional ride-sharing platforms that connect drivers with passengers, UniPool specifically targets the student community scenario where students collectively book a cab and share the cost rather than individual students owning vehicles.



2.1.1 Product Perspective

UniPool is a new, self-contained product specifically designed for the IIT Kanpur context. It is not a follow-on member of an existing product family nor a replacement for any existing system. The application fills a unique need within the campus community by providing a structured, reliable, and trust-based platform for ride coordination.

The system architecture consists of the following major components and their interconnections:

- **User authentication:** Secure login system using credentials or single sign-on to verify users and maintain the integrity of the platform.
- **Ride/Item Reporting:**"Leaders" can post detailed travel plans, including destination and time, while "Poolers" can search and join existing listings.

- **Search and filter options:**Users can filter available rides based on criteria such as destination, date, or time to find relevant matches.
- **Notifications & Chat:** Real-time updates keep users informed of status changes, supported by an integrated chat interface for coordination.
- **Verification & Trust:** A tracking system for ride history serves as a trust score, ensuring a safe and reliable experience for the community.

This product is ideal for high-density environments like college campuses, office buildings, or residential complexes where recurring travel and coordination needs are frequent.

2.2 Product Functionality

This product is designed to facilitate seamless carpooling among students while building a trusted and efficient transportation network. It includes the following features:

- User authentication and account management
- Profile creation and personalized management
- Ride creation and posting (Leader Mode)
- Ride discovery and location filtering (Pooler Mode)
- Trust verification through profile statistics
- Real-time integrated communication system
- Comprehensive ride management for leaders

2.3 Design and Implementation Constraints

The following constraints apply to the design and implementation of the UniPool application and shall be adhered to throughout the development process:

2.3.1 Database Storage

All user profiles, ride information, chat messages, and activity logs shall be stored exclusively in Cloud Firestore, which is Google Firebase's NoSQL document database.
The database shall ensure data consistency, integrity, and support real-time data synchronization across all connected clients.
Profile pictures shall be stored in Firebase Storage with appropriate size limitations to manage storage costs and ensure reasonable upload/download times.

2.3.2 User Authentication

Users must log in using their correct usernames and passwords to access their accounts and perform actions like posting or claiming items. Secure authentication protocols (e.g., OAuth2 or token-based systems) may be implemented to protect user credentials.

2.3.3 Connectivity

The application shall require an active and stable internet connection for all operations. The system shall not provide offline functionality; all features require real-time connectivity to Firebase services.The application shall handle internet connectivity loss gracefully by displaying clear, user-friendly error messages informing users that an internet connection is required. Network

interruptions during critical operations (ride posting, profile updates) shall be detected and appropriate error messages shall be shown to users.

2.3.4 Performance and Response Time

Response time for loading the ride feed shall not exceed 3 seconds under normal network conditions (defined as 4G mobile data or standard Wi-Fi connection). Chat message delivery latency shall be maintained below 1 second to ensure real-time communication experience. The application shall load user profiles, including profile pictures, within 2 seconds. The system shall handle concurrent usage by up to 500 active users without performance degradation.

2.3.5 Authentication and Security

User authentication shall be managed exclusively through Firebase Authentication using email and password methodology.Passwords shall be encrypted and secured using Firebase Authentication's built-in security protocols; no custom password encryption shall be implemented. Access to ride details and chat functionality shall be restricted to authenticated users only.Users shall remain authenticated until they explicitly log out or uninstall the application.

2.3.6 User Interface

The user interface shall be designed to operate efficiently on mobile devices with screen sizes ranging from 5 inches to 6.5 inches.The application shall support portrait orientation primarily, with consideration for landscape mode in future versions.All interactive elements shall be sized appropriately for touch-based interaction with minimum touch target sizes of 44x44 pixels.

2.3.7 Skill and Accessibility

The application shall feature an intuitive user interface requiring only basic familiarity with mobile applications.Users shall not require technical expertise to utilize any feature of the platform.The application shall be designed with simplicity in mind, ensuring that students from all academic backgrounds can use it effectively.

2.3.8 Battery and Resource Optimization

The app must be designed to minimize resource consumption, ensuring it operates efficiently without draining device battery or overloading system memory.

2.3.9 Error Handling and Recovery

The app should provide clear and informative error messages for issues like login failures, server unavailability, or incorrect user input. Automatic recovery mechanisms should handle minor disruptions (e.g., temporary network loss).

2.4 Assumptions and Dependencies

This project relies on several core assumptions and external factors to ensure successful operation within the IIT Kanpur community:

Core Assumptions

- **User Readiness:** Students possess Android devices, valid emails, and basic mobile literacy.
- **Community Integrity:** Users provide truthful profiles, act in good faith, and manage payments externally.
- **Operational Environment:** Reliable campus internet and organic user adoption for effective matching.

Key Dependencies

- **Google Firebase:** Essential for backend storage, authentication, and real-time synchronization.
- **Technical Infrastructure:** Dependent on Flutter framework stability and Android OS compatibility.
- **Connectivity:** Continuous internet access is mandatory for all application features.
- **Development Tools:** Relies on Android Studio, Flutter SDK, and Firebase Console for maintenance.

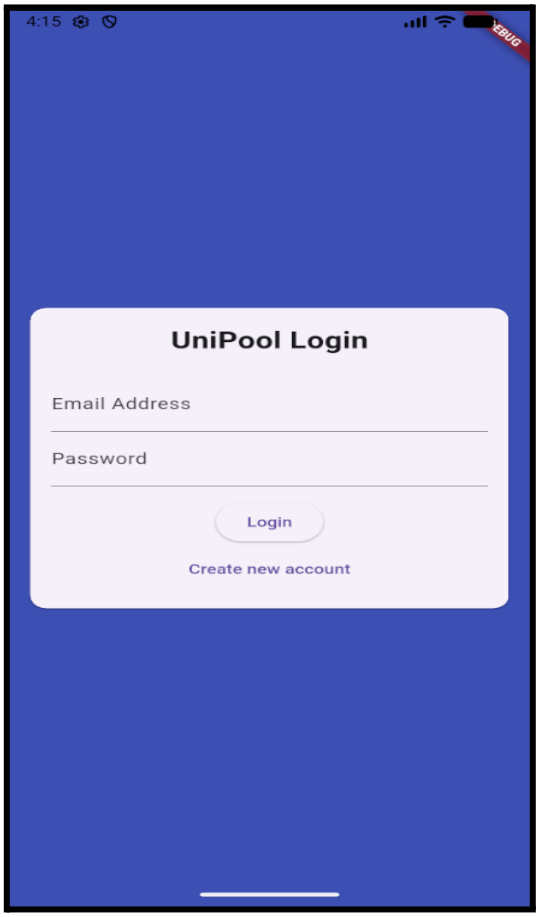
3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The user interface (UI) for the application is designed to provide an intuitive and interactive experience for users. Below are the logical characteristics of each interface and a description of how users will interact with the application.

3.1.1.1 Login Screen



Logical Characteristics:

- Users can log in using their credentials or register for a new account.
- The interface features input fields for username, password, and optional two-factor authentication.

User Interaction:

- Users input their IITK credentials and tap the Login button.

3.1.1.2 Registration Screen:

Logical Characteristics:

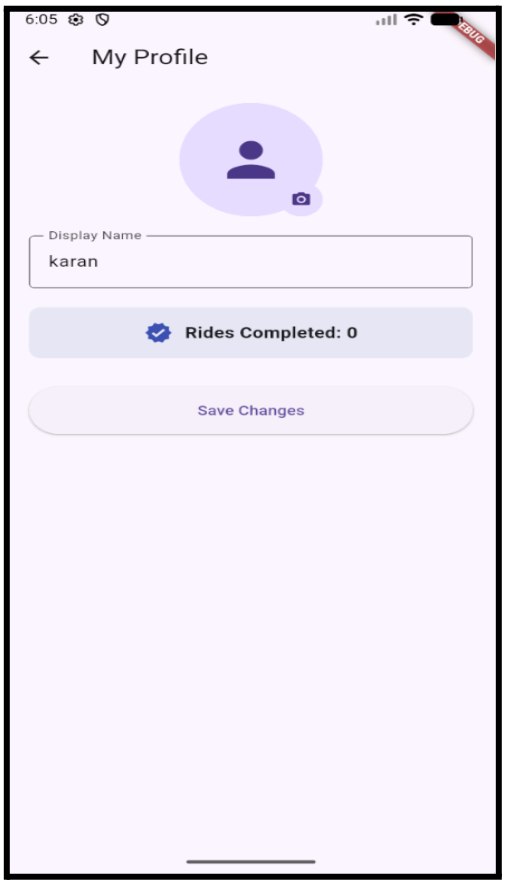
- The registration screen shall provide input fields for email address, password, and password confirmation with email format validation.
- The system shall display password strength indicators to guide users in creating secure passwords.
- The registration screen shall include a prominently displayed Create Account button and a link to return to the Login screen.

User Interaction:

- Users enter a valid email address, create and confirm a secure password, and tap Create Account to register.

- Upon successful registration, users are redirected to the Profile Creation screen, and any validation errors are displayed inline with clear guidance

3.1.1.3 Profile Creation Screen



Logical Characteristics:

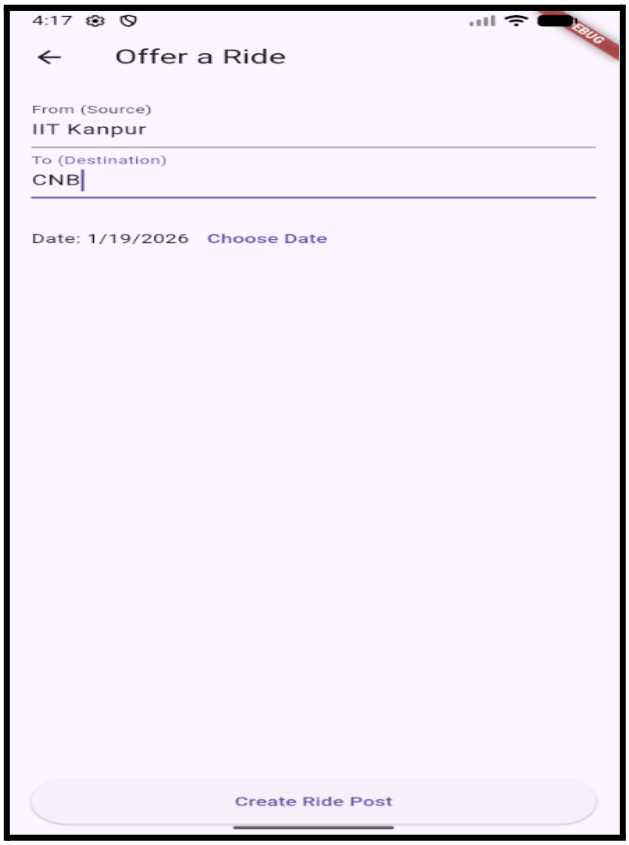
- This screen shall appear after successful registration or when a user has not completed profile setup.
- An upload area for profile picture shall be prominently displayed with a placeholder image.
- An input field for "Display Name" shall be provided.
- A "Rides Completed: 0" display shall be shown but not editable.

User Interaction:

- Users tap the profile picture area to upload an image from their device gallery or camera.
- Users enter their preferred display name.
- Users tap "Save Profile" to complete profile creation.
- Upon successful save, users are navigated to the Home Screen.

.

3.1.1.4 Leader Workflow- Create Ride Screen



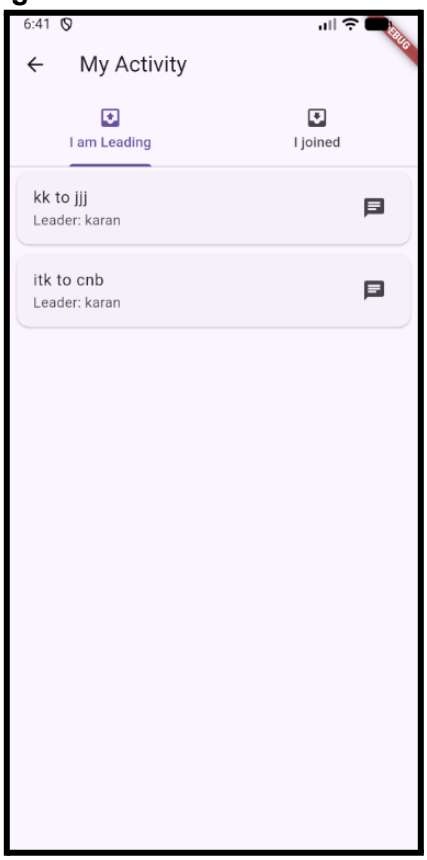
Logical Characteristics:

- Provide text fields for both **Source** and **Destination** addresses.
- Enable a **Date picker** and **Time picker** that trigger upon tapping their respective fields.
- Keep the **"Post Ride"** button disabled until all four fields are populated.
- Include a **"Cancel"** option to allow users to discard the entry at any time.
- Ensure a seamless transition from data entry to ride publication or dismissal.

User Interaction:

- Users specify **Source** and **Destination** locations and select the **Date/Time** via pickers.
- The **"Post Ride"** button becomes active once all fields are complete.
- Upon tapping "Post Ride," the system **saves the listing** to the database and generates a confirmation.
- The user is automatically **navigated back to the Home Screen** after a successful post.

3.1.1.5 User Workflow- Manage Rides-Chats Screen



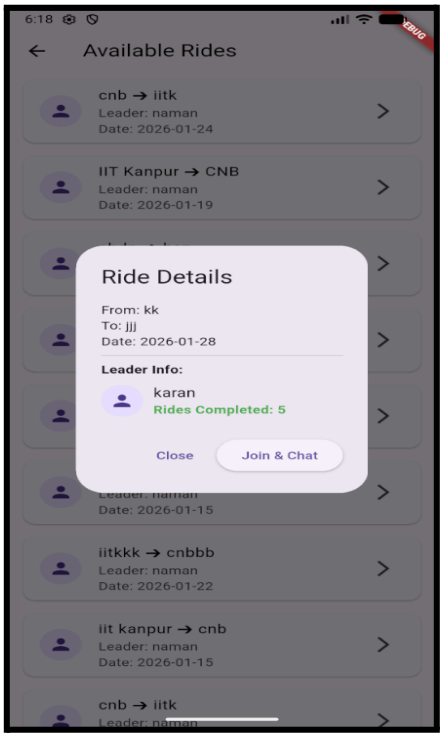
Logical Characteristics:

- A list of rides created/joined by the current user shall be displayed.
- Each ride card shall show source, destination, date, time, and a button to chat.
- A "Delete" button shall be available for each ride.
- Empty state messaging shall be displayed if no rides are posted.

User Interaction:

- Users view all their posted/joined rides.
- Users tap "Delete" on a ride card to remove the ride or chat button to view the chat.
- A confirmation dialog appears before deletion.
- Upon confirmation, the ride is removed from the database.

3.1.1.6 Pooler Workflow- Browse Rides Screen:



Logical Characteristics:

- A scrollable list of all available rides shall be displayed in reverse chronological order.
- Each ride card shall display: Source, Destination, Date, Time, and the Leader's display name.
- Filter options for Source and Destination shall be available at the top.
- A search icon or filter icon shall provide access to filtering functionality.
- Empty state messaging shall be displayed if no rides match the filters.

User Interaction:

- Users scroll through available rides.
- Users tap the filter icon to access filtering options.
- Users enter source or destination in filter fields to narrow results.
- Users tap on a ride card to view details or the Leader's profile.
- Users tap "Request/Chat" to initiate communication with the Leader.

3.1.1.7 Chat List Screen- On My Activity Screen:

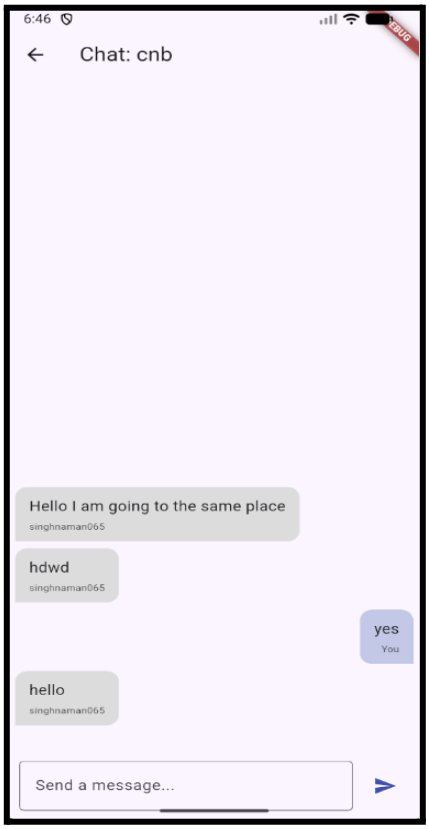
Logical Characteristics:

- A list of all ongoing conversations shall be displayed.
- Each conversation entry shall show the other user's name, and a preview of the last message.
- Conversations shall be sorted by most recent activity.
- Unread message indicators shall be displayed when applicable.
- Empty state messaging shall be shown if no conversations exist.

User Interaction:

- Users scroll through their conversation list.
- Users tap on a conversation to open the chat interface.

3.1.1.8 Chat Interface Screen:



Logical Characteristics:

- The chat screen shall display messages in chronological order, with sent messages right-aligned and received messages left-aligned with the sender's username.
- The system shall provide a fixed text input field with an adjacent send button at the bottom of the screen.
- The chat header shall display the other user's name for clear identification.

User Interaction:

- Users scroll to view message history, type messages in the input field, and tap Send to deliver messages.
- Messages appear in real time with under 1 second latency, and users can navigate back to exit the conversation.

3.1.2 Hardware Interfaces

The UniPool application shall interact with the following hardware components:

- **Android Smartphones and Tablets:** The primary devices used by users to access the application. The application shall support devices with screen sizes from 5 inches to 7 inches and running Android 6.0 (API level 23) or higher.
- **Device Storage:** The application shall access the device's photo gallery/storage through Flutter's image picker plugin to allow users to select existing images for their profile pictures.

- **Network Hardware:** The application shall utilize the device's Wi-Fi or cellular network interface cards to establish and maintain internet connectivity for all backend communications.

- **Touchscreen Display:**The application shall utilize the device's capacitive touchscreen for all user interactions, requiring touch target sizes of at least 44x44 pixels for interactive elements.

These interfaces ensure that the application functions seamlessly across various devices, providing a consistent and user-friendly experience.

3.1.3 Software Interfaces

The UniPool application shall interface with the following software components and services:

Operating System Interface:

Android Operating System: The application shall run on Android OS version 6.0 (Marshmallow, API level 23) or higher. The app shall request and utilize standard Android permissions including internet access, camera access (for profile pictures), and storage access (for selecting images from gallery).

Development Framework:

Flutter SDK: The application shall be developed using Flutter framework version 3.0 or higher, utilizing the Dart programming language. Flutter provides the cross-platform mobile development capabilities and native performance.

Backend Services (Google Firebase):

The database and its management systems would be deployed only on the server side. The final client application delivered would not be independent of this. We will be using Firebase for this.

3.2 Functional Requirements

3.2.1 F1: User Authentication System

The login screen will allow users to create and manage their personal accounts for the app, ensuring secure access to the platform. In particular, the following functionalities are provided:

- **F1.1: User Registration**
    1. The system shall allow new users to create accounts using email and password.
    2. The system shall validate that the entered email follows standard email format (contains @ symbol and domain).
    3. The system shall require passwords to be at least 8 characters in length.
    4. The system shall confirm password entry by requiring users to enter the password twice.
    5. The system shall display appropriate error messages if email format is invalid or passwords do not match.
    6. Upon successful registration, the system shall create a user record in Firebase Authentication.
    7. The system shall automatically log in the user after successful registration.

- **F1.2: User Login**
    1. The system shall allow registered users to log in using their email and password.
    2. The system shall verify credentials against Firebase Authentication database.
    3. The system shall display an error message if credentials are incorrect.
    4. Upon successful login, the system shall maintain the user session until explicit logout or app uninstallation.
    5. The system shall navigate users to the home screen after successful authentication.

- **F1.3: Session Management**
    1. The system shall maintain user authentication state across app restarts.
    2. The system shall allow users to remain logged in until they explicitly choose to log out.
    3. The system shall provide a "Logout" option in the application menu.
    4. Upon logout, the system shall clear the authentication session and return users to the login screen.

- **F1.4: Account Recovery**
    1. The system shall provide a password reset mechanism for users who forget their passwords.
    2. The system shall send password reset emails to the registered email address using Firebase Authentication.

3.2.2 F2: Profile Management System

The system shall provide comprehensive profile management functionality enabling users to create and maintain their personal profiles.

- **F2.1: Profile Creation**

    1. The system shall require users to create a profile immediately after registration.
    2. The system shall allow users to upload a profile picture from their device gallery or capture one using the camera.
    3. The system shall require users to set a display name with a minimum of 2 characters and maximum of 30 characters.
    4. The system shall initialize the "Rides Completed" counter to zero for new profiles.
    5. The system shall store profile pictures in Firebase Storage and save the download URL in the user's Firestore document.

- **F2.2: Profile Viewing**

    1. The system shall allow users to view their own profile at any time.
    2. The system shall allow users to view other users' profiles by tapping on their names in ride listings or chat interfaces.
    3. Profile views shall display the user's profile picture, display name, and rides completed count.
    4. The system shall prominently display the "Rides Completed" statistic as a trust indicator.

- **F2.3: Profile Editing**

    1. The system shall allow users to edit their display name at any time.
    2. The system shall allow users to update their profile picture at any time.
    3. The system shall not allow users to manually edit their "Rides Completed" count (this is system-managed).

- **F2.4: Rides Completed Tracking**

    1. The system shall automatically increment a user's "Rides Completed" counter when they successfully complete a shared ride.
    2. The system shall increment the counter for both Leaders and Poolers upon ride completion.
    3. The counter shall serve as a quantitative trust score visible to all users.

3.2.3 F3: Ride Creation and Management (Leader Functionality)

The system shall provide Leaders with comprehensive functionality to create, manage, and delete ride postings.

- **F3.1: Ride Posting**

    1. The system shall allow authenticated users to switch to "Leader" mode from the home screen.
    2. The system shall provide a ride creation form with the following required fields: Source, Destination, Date, and Time.
    3. The system shall validate that all fields are filled before allowing ride submission.
    4. The system shall validate that the selected date is not in the past.
    5. Upon successful submission, the system shall create a ride document in Firestore containing all ride details plus the Leader's UID, display name, and profile picture URL.
    6. The system shall assign a unique ride ID to each posted ride.
    7. The newly posted ride shall become immediately visible to all Poolers browsing rides.

- **F3.2: Ride Visibility**

    1. All rides posted by a Leader shall be visible in the Leader's "My Rides" section.
    2. All active rides shall be visible to Poolers in the browserides interface
    3. Ride listings shall display source, destination, date, time, and Leader information.

- **F3.3: Ride Deletion**

    1. The system shall allow Leaders to delete their posted rides at any time before the ride date.
    2. The system shall provide a delete button or option for each ride in the Leader's ride management interface.
    3. The system shall display a confirmation dialog before permanently deleting a ride.
    4. Upon confirmation, the system shall remove the ride document from Firestore.
    5. Deleted rides shall immediately disappear from all Poolers' browse interfaces.

3.2.4 F4: Ride Discovery and Filtering (Pooler Functionality)

The system shall provide Poolers with comprehensive functionality to discover and filter available rides.

- **F4.1: Ride Browsing**

    1. The system shall allow authenticated users to switch to "Pooler" mode from the home screen.
    2. The system shall display a complete list of all available rides posted by Leaders.
    3. Rides shall be displayed in reverse chronological order (most recent first) by default.
    4. Each ride listing shall show source, destination, date, time, and the Leader's display name.
    5. The system shall load and display rides within 3 seconds under normal network conditions.

- **F4.2: Ride Filtering**

    1. The system shall provide filter options for Source and Destination fields.
    2. The system shall allow Poolers to enter a source location to filter rides.
    3. The system shall allow Poolers to enter a destination location to filter rides.
    4. The system shall support filtering by source only, destination only, or both simultaneously.
    5. Filtered results shall update in real-time as users type or modify filter criteria and message displayed if no rides match the applied filters.

- **F4.3: Ride Details Viewing**

    1. The system shall allow Poolers to tap on any ride card to view complete ride details.
    2. The ride details screen shall display all information provided by the Leader including source, destination, date, time, and Leader information.
    3. The system shall provide clear navigation options to return to the browse rides screen.

3.5.5 F5: Trust Verification System

The system shall provide trust verification features allowing Poolers to assess Leader credibility before joining rides.

- **F5.1: Leader Profile Access**

1. The system shall display the Leader's name as a clickable element in ride listings and ride details screens.
2. The system shall navigate users to the Leader's full profile when their name or profile picture is tapped.
3. The system shall retrieve and display the Leader's profile information from Firestore in real-time.

- **F5.2: Trust Indicators Display**

1. The system shall prominently display the Leader's "Rides Completed" count on their profile.
2. The system shall display the Leader's profile picture to provide visual identity verification.
3. The system shall display the Leader's display name for identification.
4. All trust indicators shall be clearly visible and easy to read.

- **F5.3: Decision Support**

1. The profile view shall help Poolers make informed decisions about ride-sharing safety and reliability.
2. Higher "Rides Completed" counts shall indicate more experienced and potentially more trustworthy Leaders.
3. The system shall make trust information accessible before any commitment or communication is initiated.

3.6.6 F6: Real-Time Communication System

The system shall provide comprehensive real-time chat functionality enabling communication between Leaders and Poolers.

- **F6.1: Chat Initiation**

1. The system shall provide a "Request/Chat" button on ride details screens.
2. The system shall allow Poolers to initiate chat conversations with Leaders by tapping the "Request/Chat" button.
3. Upon chat initiation, the system shall create a chat conversation document in Firestore if one does not already exist.
4. The system shall navigate users to the chat interface immediately after initiation.

- **F6.2: Message Sending**

1. The system shall provide a text input field for users to compose messages.
2. The system shall allow messages up to 500 characters in length.
3. The system shall provide a send button to deliver typed messages.
4. Upon tapping send, the system shall save the message to Firestore with sender UID, timestamp, and message text.
5. The system shall display a sending indicator while the message is being transmitted.

- **F6.3: Real-Time Message Delivery**

    1. The system shall deliver messages to the recipient with less than 1 second latency under normal network conditions.
    2. The system shall utilize Firestore's real-time listeners to push new messages to all participants instantly.
    3. New messages shall appear automatically in the chat interface without requiring manual refresh.

- **F6.4: Message Display**

    1. The system shall display messages in chronological order with newest messages at the bottom.
    2. Messages sent by the current user shall appear right-aligned with distinct visual styling.
    3. Messages received from other users shall appear left-aligned with the sender's profile picture thumbnail.
    4. Each message shall display a timestamp indicating when it was sent.

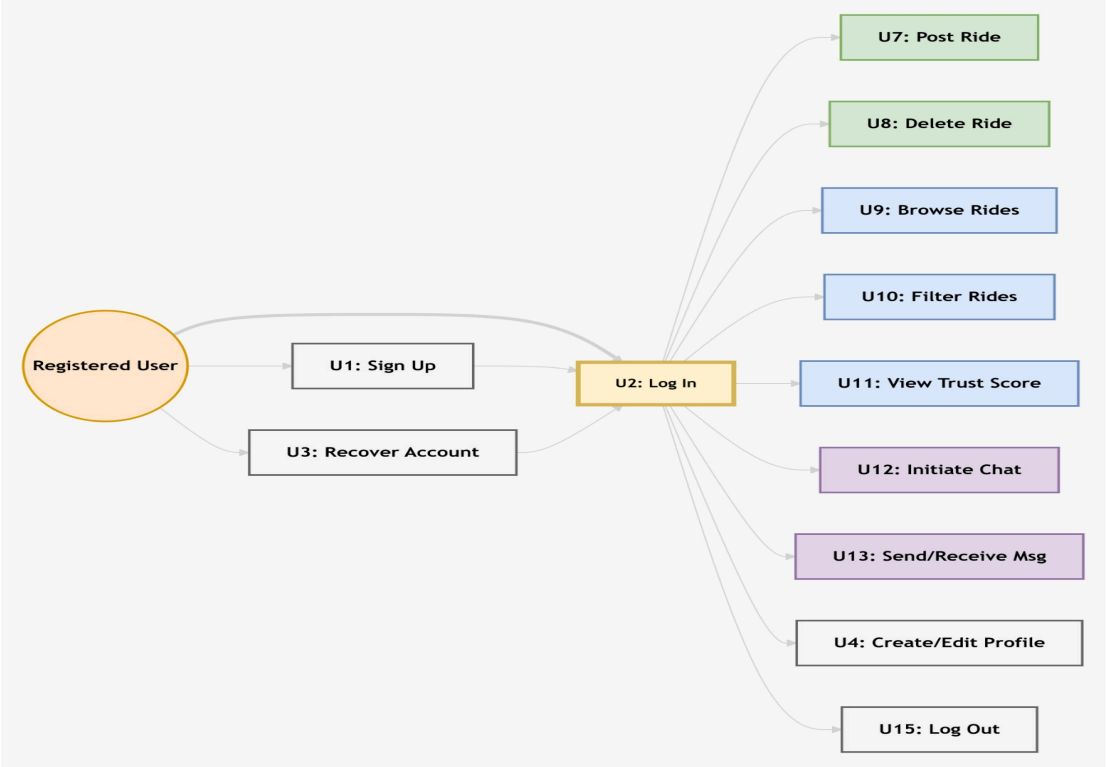
- **F6.5: Chat List Management**

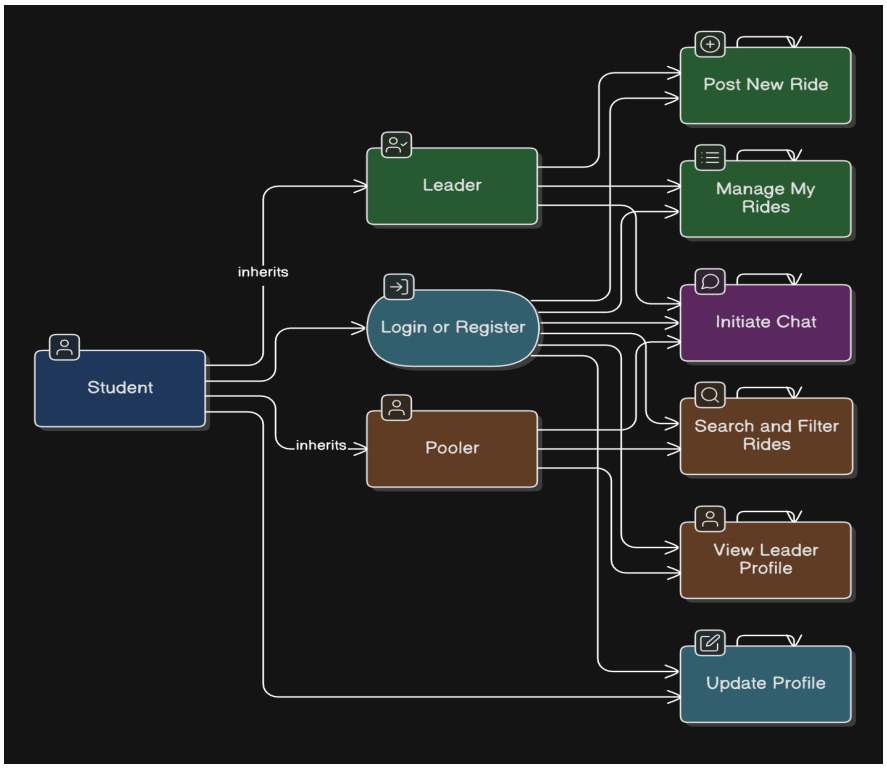
    1. The system shall maintain a list of all active conversations for each user.
    2. The chat list shall display conversation previews showing the other participant's name, profile picture, and last message snippet.
    3. Conversations shall be sorted by Leader’s chats or Pooler’s chats.
    4. The system shall allow users to access any conversation from the chat list.

- **F6.6: Chat Accessibility**

    1. Chat functionality shall be available only to authenticated users.
    2. Chats shall be private between the two participants (Leader and Pooler).
    3. The system shall not allow group chats in this version.

3.3 Use Case Model





3.3.1 U1: Sign Up

Author	Anubhav Rana
Purpose	To allow new users to create an account on the UniPool platform using their email address and a secure password, enabling them to access all application features.
Requirements Traceability	F1
Priority	High.
Preconditions	● The user must have the UniPool application installed on their Android device and have a valid email address. ● The app must not be signed in to. ● The app must have access to the Internet.
Postconditions	An account is created for the user with the specified permission level.
Actors	Members of the campus community.
Exceptions	● The entered email address may not be in valid format. The system shall display an error message: "Please enter a valid email address” ● Email verification may fail. An account will not be created. ● An account may already exist for the given email address. A new account will not be created. ● The user is not authorised to sign up with the specified permission level. An error message will be displayed. ● Internet access may not be available. An error message will be displayed.
Includes	None.
Notes	Email verification through sent emails is not implemented in the current version but may be added in future iterations

3.3.2 U2: Log In

Author	Pavi Agarwal
Purpose	To log in to the app using the credentials supplied during sign-up.
Requirements Traceability	F1
Priority	High.
Preconditions	● The user must already have signed up. ● The app must not be logged in to.

	● The app must have access to the Internet.
Postconditions	The user is navigated to the Home Screen.
Actors	Registered users.
Exceptions	Internet access may not be available. An error message will be displayed.The email address may not exist in the system. The system shall display an error message: "No account found with this email address."
Includes	U1
Notes	The application maintains persistent authentication, meaning users stay logged in across app restarts.

3.3.3 U3: Recover Account

Author	Gagandeep Kolambe
Purpose	To reset the account password if the user forgets the password set.
Requirements Traceability	F1
Priority	Medium.
Preconditions	● The user must already have signed up. ● The app must not be logged in to. ● The app must have access to the Internet. ● The user must have access to the email address associated with the account.
Postconditions	The user is prompted to set a new password. Alternatively, a system-generated password is set, and the user is redirected to the login page.
Actors	Registered users.
Exceptions	Internet access may not be available. An error message will be displayed.
Includes	U1
Notes	To authenticate before password reset, the user may be asked to enter an OTP sent on the registered email address. Alternatively, a new password is generated at random and set for the account; thereafter, it is emailed to the registered email address.

3.3.4 U4: Create Profile

Author	Alankrit Gupta, Aryan Nigam
Purpose	To enable newly registered users to set up their personal profile by uploading a profile picture and setting a display name, establishing their identity within the UniPool community.
Requirements Traceability	F2
Priority	High.
Preconditions	● The app must be logged in to. ● The app must have access to the Internet.
Postconditions	The user profile is created/updated with the new details. The change is propagated to the server. The user is navigated to the Home Screen and can now access all application features
Actors	Registered users.
Exceptions	Internet access may not be available. An error message will be displayed. The user may not grant camera permission. The system shall provide alternative option to select from gallery or skip photo upload temporarily. The display name may contain invalid characters
Includes	U2
Notes	

3.3.5 U5: View Profile

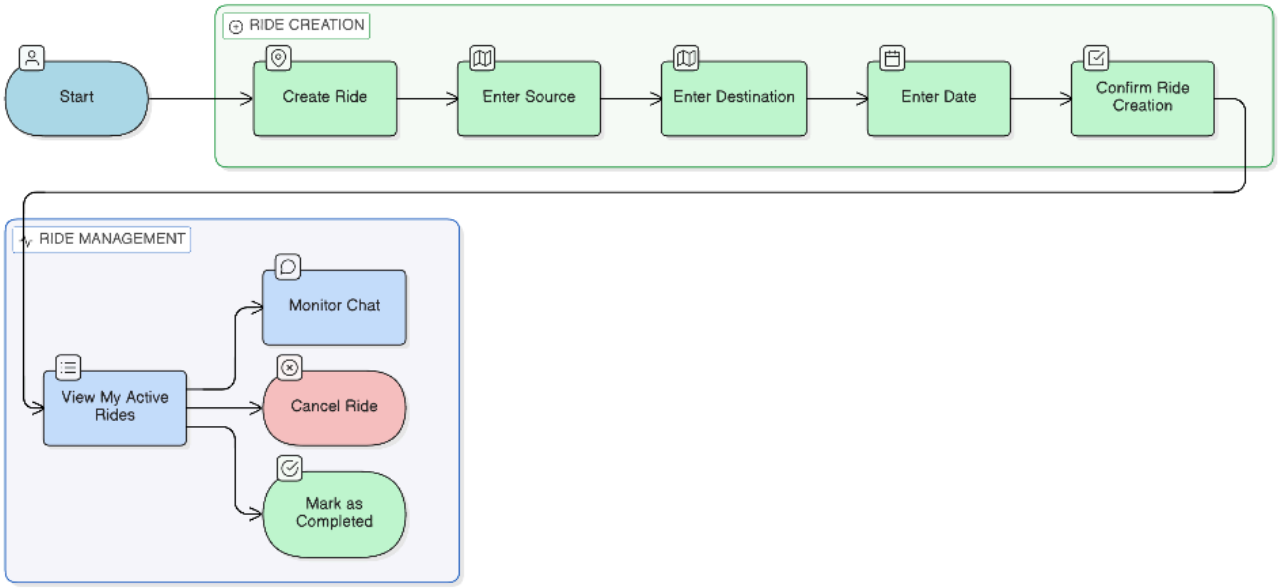
Author	Bhavishya Gupta, Harshit Gupta
Purpose	To allow users to view complete profile information.
Requirements Traceability	F2
Priority	High.
Preconditions	● The app must be logged in to. ● The app must have access to the Internet.
Postconditions	The requested profile information is retrieved from Cloud Firestore.The profile picture, display name, and rides completed count are displayed
Actors	Registered users.

Exceptions	● Internet access may not be available. An error message will be displayed. ● The server may not be online. An error message will be displayed. The photo may fail to load.
Includes	U2
Notes	● The "Rides Completed" count serves as a trust indicator and cannot be manipulated by users.

3.3.6 U6: Edit Profile

Author	Aalok
Purpose	To allow users to update their display name or profile picture after initial profile creation, maintaining accurate and current profile information..
Requirements Traceability	F2
Priority	Medium.
Preconditions	● The app must be logged in to. ● The app must have access to the Internet.
Postconditions	The updated profile information is immediately reflected across all parts of the application.Other users see the updated information in ride listings and chat interfaces.
Actors	Registered users.
Exceptions	● Internet access may not be available. An error message will be displayed.The new display name field may be left empty. The system shall display an error message: "Display name cannot be empty. ● The server may not be online. An error message will be displayed.
Includes	U2
Notes	● The "Rides Completed" count cannot be edited by users as it is system-managed. ● Changes propagate to all existing ride listings and chat conversations automatically.

3.3.7 U7: Post Ride (Leader)



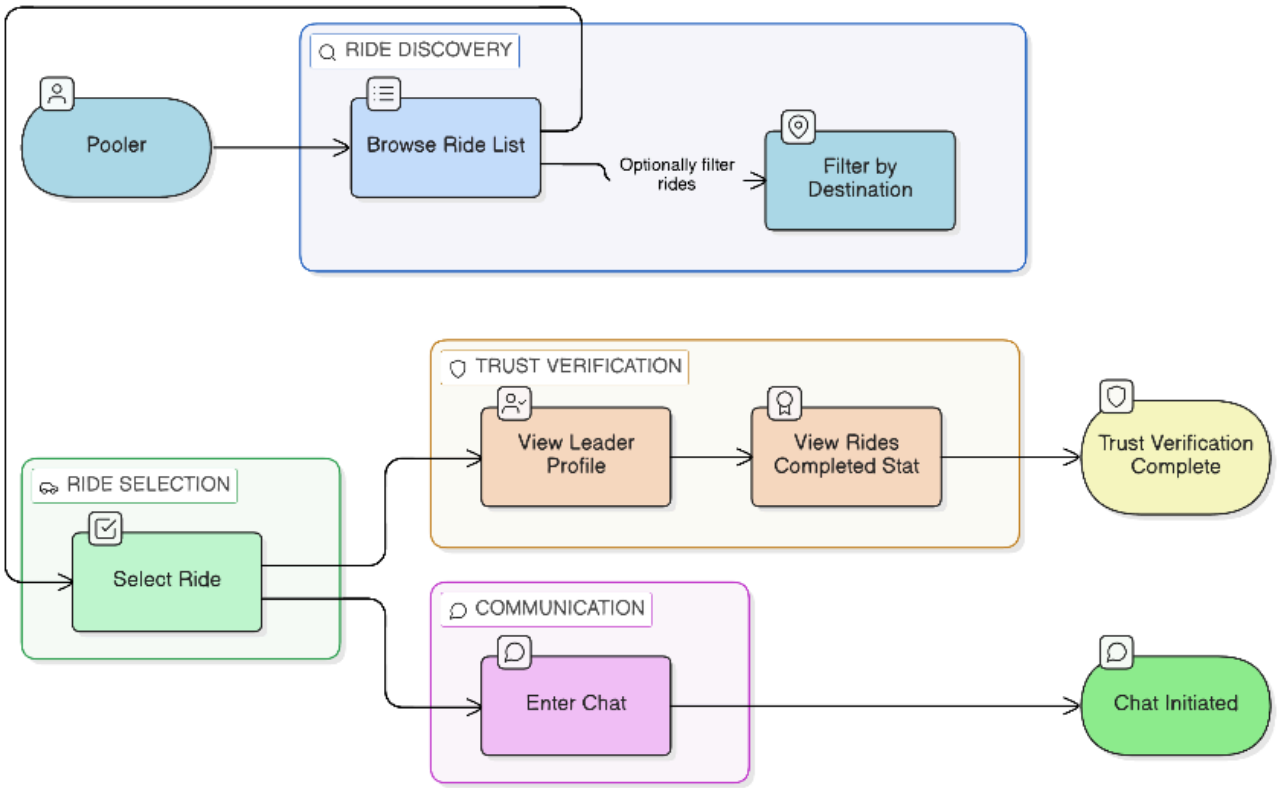
Author	Naman Mohan SIngh
Purpose	To enable users acting as Leaders to create and post new ride listings with complete travel details, making them available for Poolers to discover and join.
Requirements Traceability	F3
Priority	High.
Preconditions	• The app must be logged in to. • The app must have access to the Internet. • The user must select "Be a Leader" mode from the home screen.
Postconditions	• A new ride document is created in Cloud Firestore with a unique ride ID.The ride includes all entered details: source, destination, date, time, plus the Leader's UID, display name, and profile picture URL. • The ride becomes immediately visible to all Poolers browsing the rides list. The ride appears in the Leader's "My Rides" management section. The user is navigated back to the Home Screen.
Actors	Registered users.
Exceptions	• Internet access may not be available. An error message will be displayed. • The server may not be online. An error message will be displayed, or the request will be queued. • Any required field may be left empty, the selected date may be in the past. The system shall display an error message.

Includes	U2
Notes	Ride capacity (number of available seats) is not specified in this version but could be added in future iterations.The system does not automatically delete rides after the specified date/time has passed, requiring manual deletion by the Leader.

3.3.8 U8: Delete Ride (Leader)

Author	Karan Kumar
Purpose	To allow Leaders to remove their posted rides when plans change or the ride is no longer available, maintaining accurate ride availability information.
Requirements Traceability	F3
Priority	Medium.
Preconditions	The user must be logged into the application.The user must have previously posted at least one ride (U6 completed).
Postconditions	The ride document is permanently deleted from Cloud Firestore.The ride is removed from the Leader's "My Rides" list.
Actors	Registered users.
Exceptions	Internet connection may be unavailable. The system shall display an error message.
Includes	U7
Notes	A confirmation dialog prevents accidental deletions.

3.3.9 U9: Browse Available Rides (Pooler)



Author	Naman Mohan Singh
Purpose	To enable users acting as Poolers to view a comprehensive list of all available rides posted by Leaders, facilitating ride discovery and selection.
Requirements Traceability	F4
Priority	High.
Preconditions	• The app must be logged in to.The user must select "Be a Pooler" mode from the home screen. • The app must have access to the Internet.
Postconditions	A complete list of available rides is retrieved from Cloud Firestore.Rides are displayed in reverse chronological order (most recent first).
Actors	Registered users.
Exceptions	• Internet access may not be available. An error message will be displayed.

	• No rides may currently be available. The system shall display a friendly message: "No rides available at the moment.
Includes	U2
Notes	Sort options (by date, by proximity, etc.) could be added in future versions.

3.3.10 U10: Filter Rides by Source/Destination (Pooler)

Author	Karan Kumar
Purpose	To enable Poolers to narrow down the list of available rides by specifying source and/or destination criteria, making it easier to find rides matching their travel plans.
Requirements Traceability	F4
Priority	High.
Preconditions	• The user must be logged into the application. • Rides must be available in the system to filter.
Postconditions	The ride list updates in real-time to show only rides matching the filter criteria.If no rides match the criteria, appropriate messaging is displayed.
Actors	Registered users.
Exceptions	No rides may match the filter criteria. The system shall display a message.
Includes	None.
Notes	Filter matching is currently case-sensitive; case-insensitive matching would improve user experience.

3.3.11 U11: View Leader Profile (Pooler)

Author	Naman Mohan Singh
Purpose	To enable Poolers to access and review a Leader's complete profile including trust indicators before deciding to join a ride, facilitating informed decision-making and trust verification.
Requirements Traceability	F5

Priority	High.
Preconditions	● The app must be logged in to. ● The app must have access to the Internet.The user must be viewing a ride listing or ride details.
Postconditions	Leader's profile is displayed. A "Chat" button is available to initiate communication. - The user can make an informed decision about joining the ride.
Actors	Registered users.
Exceptions	● Internet access may not be available. An error message will be displayed. ● The server may not be online. An error message will be displayed.
Includes	U2
Notes	

3.3.12 U12: Initiate Chat and Recent Chats

Author	Naman Mohan Singh
Purpose	To enable Poolers to start a direct conversation with Leaders regarding specific rides, facilitating coordination without exchanging personal contact information.
Requirements Traceability	F6
Priority	High.
Preconditions	● The user must be logged into the application. ● The user must be viewing a ride's details or in the chat list. ● The application must have internet connectivity. ● Both the Leader and Pooler must have completed profiles.
Postconditions	The user is navigated to the chat interface. The conversation appears in both users' chat lists.The chat is ready to send and receive messages.
Actors	Registered users.
Exceptions	● Internet access may not be available. An error message will be displayed. ● The server may not be online. An error message will be displayed.
Includes	U2

Notes	The system prevents duplicate conversations between the same two users for the same ride.

3.3.13 U13: Send Message

Author	Naman Mohan Singh
Purpose	To allow users to compose and send text messages within an active chat conversation, enabling detailed discussion of ride details and coordination.
Requirements Traceability	F6
Priority	High.
Preconditions	● The app must be logged in to. A chat conversation must be initiated (U12 completed) ● The app must have access to the Internet.
Postconditions	The message appears in the sender's chat interface immediately.The text input field is cleared and ready for the next message.
Actors	Registered Users.
Exceptions	● Internet access may not be available. An error message will be displayed. ● The message text field may be empty. The system shall disable the send button when no text is entered
Includes	U2, U6
Notes	Messages support text only in this version; future versions could support images or location sharing.

3.3.14 U14: Receive Message

Author	Naman Mohan Singh
Purpose	To customise the display theme of the app.
Requirements Traceability	F6
Priority	High.
Preconditions	The app must be logged in to. A chat conversation must exist. The other participant must send a message (U13).

Postconditions	The new message is received via Firestore real-time listener. The message appears in the chat interface automatically within 1 second. The conversation moves to the top of the chat list (sorted by recent activity).
Actors	Registered users.
Exceptions	Internet connection may be lost. Messages will be delayed until connection is restored.
Includes	U2
Notes	Push notifications for new messages when app is closed would significantly improve user experience.

3.3.15 U15: Log Out

Author	Anubhav Rana
Purpose	Logging out of the app.
Requirements Traceability	F1
Priority	High.
Preconditions	The app must be logged in to.
Postconditions	The user is redirected to the login page.
Actors	Registered users.
Exceptions	None.
Includes	U2
Notes	A confirmation dialog could be added to prevent accidental logout.

4 Other Non-functional Requirements

4.1 Performance Requirements

The UniPool application shall meet the following performance requirements to ensure a smooth and responsive user experience:

4.1.1 General Performance Requirements

Response Time Requirement:

- The UniPool application shall provide responsive performance and efficient resource utilization under normal operating conditions. The system shall load ride listings, user profiles, and authentication screens within an acceptable time frame (<5 seconds) to ensure a smooth user experience. Chat messages shall be delivered in near real time with minimal perceptible delay.

Concurrency and Scalability:

- The system shall support multiple concurrent users, ride postings, and active chat conversations without noticeable performance degradation. Database queries shall return results promptly even as the volume of stored ride records increases.

Resource Utilization:

- The mobile application shall utilize device memory, storage, and battery resources efficiently during normal operation. Application size shall remain reasonably small (50-100 MB).

4.2 Safety and Security Requirements

User Authentication Security:

- All passwords shall be encrypted and securely stored using Firebase Authentication's built-in encryption mechanisms, which employ industry-standard algorithms.
- The system shall not store passwords in plain text at any point in the application or database.
- Failed login attempts shall be logged, and accounts may be temporarily locked after multiple consecutive failures (handled by Firebase Authentication).

Data Privacy and Protection:

- Personal user data including email addresses, display names, and profile pictures shall be stored with appropriate access controls in Cloud Firestore.
- User email addresses shall not be displayed publicly within the application; only display names shall be visible to other users.
- The system shall enforce Firebase security rules to ensure users can only read and write data they are authorized to access.

- Chat messages shall be private and accessible only to the two participants in the conversation.
- Users shall not be able to access, modify, or delete other users' data except their own.

Network Security:

- All communication between the mobile application and Firebase backend services shall occur over HTTPS encrypted connections.
- API calls to Firebase shall include authentication tokens to prevent unauthorized access.
- The system shall validate and sanitize all user input to prevent injection attacks.

Account Security:

- Users shall be required to be authenticated to access any feature beyond the login/registration screens.
- Authentication sessions shall remain active until explicit logout or app uninstallation.
- The system shall not expose user identification numbers (UIDs) in user-facing interfaces.

Data Integrity:

- The system shall ensure that the "Rides Completed" counter cannot be manipulated by users and can only be incremented through system-controlled processes.
- Ride data shall include creator information that cannot be modified after creation to maintain accountability.
- Timestamps on all data (rides, messages, profiles) shall be server-generated to prevent client-side manipulation.

4.3 Software Quality Attributes

The app must meet specific software quality attributes to ensure a robust, user-friendly, and maintainable system. Below, we outline the critical quality attributes, their definitions, and how they will be achieved.

4.3.1 Reliability

Definition:
The ability of the system to function correctly and consistently under specified conditions.

Requirements:
- The application shall maintain a 99% uptime rate when Firebase services are available.
- The system shall not lose user data (profile information, ride postings, chat messages) under normal operating conditions.
- The system shall gracefully handle Firebase service outages by displaying appropriate error messages without crashing.
- All critical operations (ride posting, profile updates, message sending) shall include error handling and retry mechanisms.

Implementation Approach:

- Implement comprehensive error handling using try-catch blocks for all Firebase operations.

- Use Firestore transactions for operations requiring data consistency.
- Implement automatic retry logic with exponential backoff for failed network operations.
- Log all errors to Firebase Crashlytics for monitoring and debugging.
- Test the application thoroughly under various network conditions including poor connectivity and complete disconnection.

4.3.2 Usability

Definition:
The ease with which users can learn, navigate, and efficiently interact with the application.

Requirements:
- New users shall be able to create an account, set up their profile, and post their first ride within 5 minutes without external assistance.
- All primary features (posting rides, browsing rides, chatting) shall be accessible within 3 taps from the home screen.
- The user interface shall follow Android Material Design guidelines for familiar and intuitive interaction patterns.
- Error messages shall be clear, specific, and provide actionable guidance to users.
- All interactive elements shall have minimum touch target sizes of 44x44 pixels for easy tapping.

Implementation Approach:
- Conduct usability testing with actual IIT Kanpur students to identify pain points and areas for improvement.
- Implement clear visual hierarchy with consistent color schemes and typography throughout the application.
- Provide helpful placeholder text and labels for all input fields.
- Use loading indicators for all operations that take more than 1 second to complete.
- Include onboarding screens or tooltips for first-time users to familiarize them with key features.

4.3.3 Maintainability

Definition:
The ease with which the software can be modified, updated, or debugged.

Requirements:
- The codebase shall follow Flutter and Dart best practices and coding conventions.
- Code shall be organized into logical modules with clear separation of concerns (UI components, business logic, data models, Firebase services).
- All functions and classes shall include documentation comments explaining their purpose and parameters.
- Bug fixes shall not require more than 2 developer-days for implementation and testing.
- New feature additions to existing modules shall not require more than 5 developer-days.

Implementation Approach:
- Use a well-defined project structure separating screens, widgets, models, services, and utilities into distinct directories.
- Implement state management using Provider or similar pattern to separate business logic from UI.

- Write reusable widget components to minimize code duplication.
- Use version control (Git) with meaningful commit messages and branch naming conventions.
- Conduct code reviews before merging changes to the main branch.
- Maintain comprehensive documentation of the codebase architecture and key components.

4.3.4 Security

Definition:
Security ensures that the app protects user data from unauthorized access and breaches.

Requirements:
- All user data (e.g., passwords, personal information) shall be encrypted.
- The system shall be tested for vulnerabilities using industry-standard penetration testing methods before deployment.

Implementation Approach:
- Monitor suspicious activity and implement automated lockout mechanisms for failed login attempts.
- Store login status tokens securely.
- Apply security patches regularly to address known vulnerabilities.

4.3.5 Portability

Definition:
The ability to transfer the software to different environments or platforms.

Requirements:
- The application shall be developed using Flutter framework to enable future portability to iOS with minimal code changes.
- Platform-specific code shall be isolated in separate modules and kept to a minimum.
- The application shall not rely on Android-specific features that cannot be replicated on iOS.
- At least 95% of the codebase shall be platform-independent Flutter/Dart code.

Implementation Approach:
- Use Flutter's platform-agnostic widgets and packages wherever possible.
- When platform-specific functionality is required, use Flutter's platform channels with abstraction layers.
- Test critical functionality on both Android emulators and physical devices of various screen sizes.
- Avoid using Android-specific libraries or plugins that lack iOS equivalents.
- Design the UI using responsive layouts that adapt to different screen sizes and orientations.

4.3.6 Adaptability

Definition:
Adaptability ensures that the app can be modified to support new features or integrate with external systems without significant redesign.

Requirements:

- The system shall support the addition of new features without requiring a complete overhaul.

Implementation Approach:

- Design the app using a microservices architecture, allowing independent updates to specific components.
- Use APIs and standardised data formats (e.g., JSON, XML) for external integrations.

Appendix A – Data Dictionary

Field Name	Data type	Description	Possible States/Values	Operations/ Functionality	Constraints/ Requirements
`uid`	`String`	Unique user identifier assigned by Firebase Authentication	Auto-generated UUID by Firebase	Used to identify users uniquely across all collections; referenced in rides and messages.	Must be unique; auto-generated ; immutable.
`email`	`String`	User's email address used for authentication.	Valid email format (e.g., student@iitk.ac .in).	Used for login; stored encrypted by Firebase Auth; not displayed publicly.	Must follow email format validation; required; unique.
`password`	`String`	User's chosen password for account security.	Alphanumeric string, minimum 8 characters.	Encrypted by Firebase Auth; used for authentication.	Minimum 8 characters; encrypted; not stored in plain text.
`displayName`	`String`	User's chosen name displayed throughout the app.	Alphanumeric string, 2-30 characters	Displayed in profiles, ride listings, chat interfaces.	Required; 2-30 characters; can contain letters, numbers, spaces.
`photoUrl`	`String(URL)`	Firebase Storage URL of user's profile picture.	Valid HTTPS URL pointing to Firebase Storage.	Retrieved and displayed in profiles, ride listings, chats.	ptional; must be valid Firebase Storage URL;max image size 5MB
`ridesCompleted`	`Integer`	Count of total rides successfully completed by user.	Non-negative integer, starts at 0.	Displayed as trust indicator; auto-incremented by system; cannot be	Default: 0; read-only for users; system-managed.

				manually edited.	
`createdAt`	`Timestamp`	Date and time when the user account was created.	Firebase Timestamp.	Used for account tracking and analytics.	Auto-generated on account creation; immutable.
`rideId`	`String`	Unique identifier for each ride posting	Auto-generated UUID by Firestore	Used to uniquely identify and reference specific rides.	Must be unique; auto-generated ; immutable.
`source`	`String`	Starting location/pickup point for the ride	Free-form text, up to 100 characters	Entered by Leader; displayed in ride listings; used for filtering	Required; max 100 characters
`destination`	`String`	Ending location/drop-of f point for the ride	Free-form text, up to 100 characters	Entered by Leader; displayed in ride listings; used for filtering.	Required; max 100 characters
`date`	`Timestamp`	Date of travel for the ride	Firebase Timestamp (date component)	Selected by Leader using date picker; displayed in ride listings.	Required; cannot be in past
`time`	`Timestamp`	Departure time for the ride	Firebase Timestamp (time component)	Selected by Leader using time picker; displayed in ride listings	Required; combined with date, cannot be in past
`Leaderid`	`String`	UID of the Leader who posted the ride	Valid user UID from users collection	References user who created ride; used to display Leader info.	Required; must match existing user; immutable after creation.

`creatorName`	`String`	Leader (denormalized for performance)	String matching Leader's displayName at time of posting	Displayed in ride listings for quick reference	Required; copied from creator's profile
`creatorPhotoUrl`	`String (URL)`	Profile picture URL of Leader (denormalized)	Valid HTTPS URL or empty string	Displayed in ride listings	Optional; copied from creator's profile
`messageId`	`String`	Unique identifier for each chat message	Auto-generated UUID by Firestore	Used to uniquely identify messages	Must be unique; auto-generated ; immutable
`conversationId`	`String`	Unique identifier for each chat conversation	Auto-generated UUID by Firestore	Used to group messages into conversations	Must be unique; auto-generated ; immutable
`senderId`	`String`	UID of user who sent the message	Valid user UID	Identifies message sender; used to format message display	Required; must match existing user
`recipientId`	`String`	UID of user who receives the message	Valid user UID	Identifies message recipient; used for routing	Required; must match existing user
`messageText`	`String`	Content of the chat message	Free-form text, up to 500 characters	Displayed in chat interface	Required; max 500 characters; cannot be empty
`timestamp`	`Timestamp`	Date and time when message was sent	Firebase Timestamp	Used for sorting messages chronologically	Auto-generated on message send; immutable

`participants`	`Array<String>`	UIDs of users in conversation	Array containing exactly 2 UIDs	Used to query conversations for specific user pairs	Required; must contain exactly 2 valid UIDs
`lastMessage`	`String`	Preview of most recent message in conversation	String (first 50 chars of last message)	Displayed in chat list for preview	Updated automatically; max 50 chars

Appendix B – Group Log

Date & Time	Venue	Minutes
07/01/25 **3:00 pm**	1st floor, Rajeev Motwani Building	Brainstormed project ideas and settled down on a cab pooling application. Decided the target platform to be Android devices. Discussed the basic features and initial scope of the application.
10/01/25 **3:00 pm**	4th floor, Rajeev Motwani Building	*A meeting was held with the course instructor* to discuss the cab pooling project, including its challenges and potential use cases. Based on the discussion, the initial idea of a live feature was revised, and the instructor advised implementing a static version for better feasibility and scope management.
13/01/25 **9:30 pm**	1st floor, Rajeev Motwani Building	Discussed core features and target user base. Identified key pain points in current student carpooling arrangements.Technology stack decision meeting. Selected Flutter for frontend development and Firebase for backend services.
17/01/25 **5:00 pm**	Hall 1,Canteen	*Meeting with TA.* Discussed the overall idea and use cases with the TA and confirmed the feasibility of the idea.
20/01/25 **3:00 pm**	4th floor, Rajeev Motwani Building	Final review of the SRS document with all team members. Commenced interface design with rough outlines developed.